

Gün 17 — Frontend-Backend Entegrasyonu ve Sonuç Gösterimi

Hedef

Gün 16'da hazırlanan sürükle-bırak arayüzünü gerçek backend'e (`/api/v1/analyze`) bağlamak: dosya seçildiğinde otomatik analiz isteği gönderilmesi, `threat_score` in görsel bir gösterge ile gösterilmesi, `analysis_summary` metninin ekranda gösterilmesi, ve `extracted_video_url` doluysa gizli videonun HTML5 `<video>` player ile oynatılması. Farklı origin'den (statik dosya sunucusu veya `file://`) çalışan `frontend/` 'in `http://127.0.0.1:8000` üzerindeki backend'e istek atabilmesi için FastAPI tarafında CORS yapılandırması da bu günün kapsamındaydı.

Yaklaşım

1. CORS (`backend/app/main.py`)

`CORSMiddleware` , `allow_origins=["*"]` ile eklendi. Proje herhangi bir kimlik doğrulama/cookie/oturum mekanizması kullanmıyor ve `frontend/` şu an build adımsız, `file://` ile doğrudan veya herhangi bir statik sunucu portundan açılabilir; bu yüzden origin'i tek bir değere sabitlemek (örn. yalnızca `http://127.0.0.1:5500`) gereksiz bir kısıtlama olurdu. Bu ayar yalnızca geliştirme/staj ortamı için uygundur — gerçek bir production dağıtımında `allow_origins` bilinen frontend origin'i ile sınırlandırılmalı (bkz. Notlar/Riskler).

2. `fetch` entegrasyonu (`frontend/app.js`)

- `API_BASE_URL = "http://127.0.0.1:8000"` sabiti eklendi; `frontend/` ile backend farklı origin'lerden servis edildiği için endpoint'e göreli (`/api/v1/analyze`) değil, tam URL ile istek atılıyor.
- Mevcut `handleFile()` akışına (dosya doğrulama → önizleme) dokunulmadan, geçerli bir dosya seçildiğinde otomatik olarak `analyzeFile(file)` çağrılıyor — ayrı bir "Analiz Et" butonu yok, Gün 16'daki "seç ve gör" UX akışıyla tutarlı.
- `analyzeFile()` : `FormData` ile `POST /api/v1/analyze` . `AbortController` kullanılarak, kullanıcı bir istek devam ederken hızlıca ikinci bir dosya seçerse önceki istek iptal ediliyor — bu olmadan, eski (yavaş) isteğin yanıtı yeni seçimin üzerine geç gelip yanlış sonucu ekrana yazabilirdi (race condition).
- Hata yönetimi backend'in şemasıyla uyumlu: `response.ok` değilse gövde JSON olarak parse edilip `detail` alanı (Gün 15'teki `HTTPException` /global exception handler çıktısı) kullanıcıya gösteriliyor; ağ hatası (backend ayakta değilse) `catch` bloğunda genel bir mesajla yakalanıyor.

- `setLoading(true/false)` çağrılarını `try/finally` ile sarılarak, hata olsa bile loading göstergesinin takılı kalması engellendi.

3. Sonuç gösterimi (`renderResults`)

- **Durum rozeti:** `polyglot_status` → "Polyglot tespit edildi" (kırmızı) / "Temiz dosya" (yeşil).
- **Tehdit skoru göstergesi:** sayısal değer (`{score}/100`) + yatay bar (`width: {score}%`). Renk eşiği: 0-33 yeşil (düşük), 34-66 sarı (orta), 67-100 kırmızı (yüksek) — bu eşikler `backend/app/pipeline.py` 'deki `compute_threat_score` ağırlıklarıyla (trailer tespiti tek başına 60 puan verdiği için polyglot dosyalar doğal olarak "orta/yüksek" bandına düşer) tutarlı seçildi.
- **Özet metni:** `analysis_summary` doğrudan bir paragrafa yazılıyor.
- **Video player:** `extracted_video_url` doluysa `<video controls>`, `src="${API_BASE_URL}${extracted_video_url}"` ile eklenip `/media/...` üzerinden statik olarak sunulan ayıklanmış videoyu oynatıyor.

4. Stil (`frontend/style.css`)

Mevcut koyu tema token sistemine (`:root` değişkenleri) iki yeni renk eklendi: `--color-success` / `--color-warning` (mevcut `--color-danger` ile birlikte üç seviyeli tehdit göstergesini kapsıyor). Yeni bir framework/kütüphane eklenmedi; `.result-badge`, `.threat-bar`, `.result-video` gibi bloklar mevcut `.dropzone` / `.file-preview` stil diliyle aynı `--radius` / `spacing` değerlerini kullanıyor.

Dosyalar

```
backend/app/main.py    # CORSMiddleware eklendi (allow_origins=["*"])
frontend/app.js        # analyzeFile(), renderResults(), AbortController, API_BASE_URL
frontend/style.css     # sonuç rozeti/bar/video player stilleri, success/warning renk tokeni
```

Test Sonuçları

Bu oturumda `claude-in-chrome` bağlı değildi (kullanıcı kurulumu bu oturum için ertelendi), bu yüzden gerçek bir tarayıcıda tıklama/sürükle-bırak etkileşimi otomatik olarak doğrulanamadı — Gün 16'daki gibi bu açıkça not düşülüyor. Bunun yerine backend + frontend birlikte ayağa kaldırılıp gerçek bir tarayıcının yapacağı isteklerle uçtan uca doğrulandı:

1. Backend `uvicorn app.main:app` ile `127.0.0.1:8000` 'de, `frontend/` ayrı bir origin'den (`python3 -m http.server 5500`) ayağa kaldırıldı — Gün 16'daki "frontend ayrı origin'den açılabilir" senaryosunun birebir aynısı.
2. `OPTIONS /api/v1/analyze` (CORS preflight) `Origin: http://127.0.0.1:5500` ile `200 OK` + `access-control-allow-origin: *` döndü.

3. `Origin: http://127.0.0.1:5500` header'ıyla gerçek bir `POST /api/v1/analyze` isteği (polyglot PNG) `201 Created` + `access-control-allow-origin: *` ile döndü; yanıt şeması `AnalyzeResponse` ile birebir uyumlu (`polyglot_status=true` , `threat_score=76` , `extracted_video_url` dolu).
4. Aynı isteğin döndürdüğü `extracted_video_url` (`/media/...mp4`), `Origin: 127.0.0.1:5500` header'ıyla `200 OK` + `content-type: video/mp4` döndürdü — `<video src>` 'in tarayıcıda gerçekten oynatılabilir olacağını doğruluyor.
5. Temiz bir JPEG (`samples/sample.jpg`) için `polyglot_status=false` , düşük `threat_score` (2) ve `extracted_video_url=null` döndü — "temiz dosya" rozetinin/yeşil bar'ın doğru koşulda tetikleneceği teyit edildi.
6. `app.js` elle satır satır gözden geçirildi (`node` kurulu olmadığı için otomatik sözdizimi kontrolü yapılamadı — Gün 16'da da aynı kısıt vardı); tüm yeni DOM/ `fetch` çağrılarının `index.html` 'deki mevcut id'lerle ve backend'in `AnalyzeResponse` şemasındaki alan adlarıyla birebir eşleştiği doğrulandı.

Kullanıcının kendi tarayıcısında backend'i çalıştırıp `frontend/index.html` 'i açarak bir polyglot ve bir temiz dosya ile son doğrulamayı yapması önerilir — bu, Gün 17'nin kabul kriterinin nihai teyididir.

Kabul Kriterleri — Durum

- [x] Bir polyglot dosya yüklendiğinde risk skoru, özet ve gömülü video player'ın tarayıcıda görüntülenmesi (curl ile CORS + şema + video erişilebilirliği doğrulandı; gerçek tarayıcı etkileşimi kullanıcı tarafından son kez teyit edilmeli)

Notlar/Riskler

- `allow_origins=["*"]` yalnızca geliştirme ortamı için uygundur; proje bir production ortamına taşınırsa (Gün 19-20 kapsamı dışında, ileriye dönük not) origin listesi bilinen frontend adresleriyle sınırlandırılmalı.
- İstemci tarafı otomatik analiz (dosya seçilir seçilmez `fetch` tetiklenmesi) UX'i basitleştiriyor ama kullanıcıya "önce gözden geçir, sonra gönder" seçeneği sunmuyor; bu, projenin kapsamındaki tek adımlı analiz akışıyla (Gün 12'deki tek endpoint tasarımı) tutarlı bir tercih.
- `AbortController` yalnızca istemci tarafında yarım kalan isteği iptal eder; backend'de zaten başlamış bir `run_pipeline` çalıştırması iptal isteğiyle durdurulmaz (event loop'u bloklamayan `asyncio.to_thread` çağrısı arka planda tamamlanmaya devam eder). Bu, projenin ölçeğinde (tek kullanıcı, kısa analiz süresi) kabul edilebilir bir sınırlama.